

09 - Seguridad

Ejercicio 09 — Seguridad ★ ★ ★ ★

Foco: auditoría de seguridad, secretos, inyección, fuga de información **Tiempo estimado:** 2 clases [← Volver al README](#)

Objetivo

Que uses la IA como **auditor de seguridad** y aprendas a distinguir una vulnerabilidad real de una alarma genérica. Cierra el TP porque cruza casi todo lo anterior (validación, auth, errores).

El problema

El proyecto tiene **varios problemas de seguridad reales** (algunos típicos de código educativo). Tu trabajo es encontrarlos con la IA, **confirmar cuáles son reales en ESTE código**, y arreglar los importantes:

- **Credenciales en el repo:** `src/configs/db-config.js` ya **no** tiene credenciales hardcodeadas — las lee del `.env` (✅ buen primer paso). Pero el problema se corrió de lugar: verificá que el `.env` esté en `.gitignore` y que **no esté commiteado**. Si las credenciales estuvieron alguna vez en un commit (en el `.env`, o en una versión vieja de `db-config.js` que tenía la contraseña escrita), **siguen en el historial de git** aunque hoy no aparezcan en el código.
 - **Fuga de información en errores:** los controllers devuelven `error.message` crudo al cliente. Un error de pg puede revelar nombres de tablas/columnas, ayudando a un atacante.
 - **Errores tragados:** los repositories hacen `catch → log → return null`. Esto oculta fallos (¿el `null` es "no hay datos" o "se cayó la base"?) y complica detectar un ataque.
 - **CORS abierto:** `app.use(cors())` sin configuración permite peticiones desde **cualquier** origen.
 - **Falta de auth** (lo resolviste en el ej. 05 — referencialo).
 - **SQL injection:** ¿está? Las queries usan `$1, $2` parametrizados → **eso está bien**. Pero verificalo: si en algún lado se concatena un string a la query, ahí sí hay inyección.
-

Qué tenés que lograr

1. Un **informe de seguridad** `prompting/entregas/09-informe-seguridad.md` listando vulnerabilidades, cada una clasificada por **severidad** (alta/media/baja) y si es **real o falsa alarma** en este código.
2. Arreglar las de severidad alta: como mínimo, **asegurar que el `.env` esté en `.gitignore` y fuera del tracking de git** (las credenciales ya salieron de `db-config.js`, pero el `.env` no debe commitearse), y **no**

filtrar `error.message` al cliente. Si alguna credencial quedó en el historial de git, dejá anotado en el informe que hay que **rotarla**.

3. Configurar CORS con una lista de orígenes permitidos.

Cómo encarar el prompting

💡 La seguridad es donde la IA tira **muchos falsos positivos** (te alarma con cosas que no aplican) y a veces **falsos negativos** (no ve lo que tenés delante). Tu pensamiento crítico es el que decide.

Prompt de auditoría:

"Actuá como un pentester revisando esta API Express + PostgreSQL. Te paso `db-config.js`, un controller, un repository y `server.js`. Listá vulnerabilidades con severidad y, para cada una, indicá si es explotable en ESTE código concreto o si es una recomendación genérica. Incluí cómo se explotaría."

⚠️ **Trampa #1 — el falso positivo de SQL injection:** la IA puede gritar "¡SQL injection!" al ver queries. Pero si usás `$1`, `$2` con `values`, **no hay inyección** (pg parametriza). Verificá vos: ¿hay concatenación de strings en alguna query? Si no, es falsa alarma — y tenés que poder explicar **por qué** los placeholders te protegen.

⚠️ **Trampa #2 — `.env` no es seguro por sí solo:** mover el secreto a `.env` ayuda **solo si** `.env` **está en** `.gitignore`. Si commiteás el `.env`, no cambió nada. Verificá el `.gitignore`. Y si ya estaba commiteado en el historial de git, la credencial **sigue ahí** aunque la borres ahora (mencionalo en el informe).

⚠️ **Trampa #3 — el secreto en el repo ya está quemado:** una vez que una contraseña estuvo en un commit público, hay que **rotarla** (cambiarla), no solo borrarla. La IA rara vez te lo dice solo.

💡 **Code review** (🟢 habilitado por EFSI): pedile a la IA un *security code review* de tu propio código después de arreglarlo, para ver si quedó algo.

Verificación del resultado

- ☐ No hay credenciales en ningún `.js` (las buscaste vos, no confiaste en la IA) — `db-config.js` lee todo del `.env`.
- ☐ `.env` está en `.gitignore` **y no está trackeado** (comprobalo con `git ls-files | grep .env`: solo debería aparecer `.env-template`).
- ☐ `.env-template` tiene la estructura de claves **sin** valores reales (es la plantilla que copian los demás).
- ☐ Los errores que ve el cliente **no incluyen** el `error.message` crudo de pg (probá forzar un error y mirá la respuesta).
- ☐ CORS está restringido a orígenes conocidos (no `cors()` pelado).
- ☐ En el informe, **cada** vulnerabilidad dice si es real o falsa alarma, con justificación.
- ☐ Clasificaste correctamente la parametrización `$1` como **protección** contra inyección, no como vulnerabilidad.

🤔 Pregunta para el oral: ¿por qué `SELECT * FROM alumnos WHERE id=$1` con `values=[id]` NO es vulnerable a SQL injection, pero `SELECT * FROM alumnos WHERE id=${id}` sí lo sería? Mostrame la diferencia en tu código.

✓ Entrega

Bitácora + informe de seguridad + commit de los arreglos. **Obligatorio:** identificá al menos **un falso positivo** que tiró la IA y explicá por qué no aplica acá. Saber descartar es parte de la habilidad.